LIBSEAT

Library Seat & Resource Management System

Product Requirements Document

BITS Pilani Hyderabad | Academic Case Study | PostgreSQL + Flask

PostgreSQL | Flask API | BITS Pilani HYD

TABLE OF CONTENTS

01 EXECUTIVE SUMMARY

Product Vision

LibSeat is a library seat and resource management system for BITS Pilani Hyderabad. It allows students to book library seats and reference books together through a web portal, while giving librarians a real-time dashboard for occupancy analytics, holiday management, and automated conflict resolution. The system is built on a PostgreSQL database with a Flask REST API backend and enforces institutional integrity at the database level.

Scope

LibSeat covers the full lifecycle of a library visit — from checking seat and book availability, creating a conflict-free booking, viewing history, to automated cleanup when resources are removed. It serves three distinct user types: students (booking), librarians (operations), and admins (data management).

0 conflicts	< 30s	Auto-reassign	1 procedure
Guaranteed by PG function	Time to find + book seat	On seat/book deletion	Entire holiday cancelled

Key Architectural Decisions

- **Database-level constraint enforcement** — Email validation, conflict detection, and reassignment logic live in PostgreSQL — not in application code — making the system robust to any frontend or backend swap
- **BEFORE DELETE triggers over application-layer guards** — Ensures no orphaned booking exists regardless of how deletion is triggered — CLI, API, or admin console
- **Composite primary keys for BookingBook_ID** — Using (student_id, book_id, start_time) prevents duplicate book allocations in the same booking slot without a separate surrogate key
- **No booking on future holidays** — The declare_holiday_and_delete_bookings() procedure is a single atomic transaction — either all next-day bookings cancel or none do

Out of Scope (V1)

- Mobile app (web portal only)
- Email/SMS notification to students on booking cancellation
- Payment or fine management
- Integration with BITS ERP student database
- Multi-campus deployment (Hyderabad campus only)

02 PROBLEM STATEMENT

"Students at BITS Pilani Hyderabad spend valuable time walking to the library only to find all seats occupied, discover their reserved reference book is already assigned to someone else, or learn their booking was silently cancelled for a holiday they weren't notified about."

Pain 1 — No Real-Time Seat Visibility

The library has multiple rooms (locations) with different seat capacities. Before LibSeat, there was no central system showing which seats were free at any given time. Students made physical trips only to find no availability, wasting 15–30 minutes per failed attempt.

- No digital visibility into seat occupancy by room or time slot
- Students cannot plan study sessions in advance
- Group study coordination requires multiple in-person scouting trips

Pain 2 — Book Reservation Conflicts

Reference books with the same ISBN may have multiple physical copies. Without a booking system, two students could reserve what they thought was 'the same book' but were actually competing for the same physical copy — leading to disputes at the library counter.

- Same-ISBN copy allocated to conflicting time slots
- No mechanism to find an alternative copy when one is removed from inventory
- Librarians resolved conflicts manually, taking 10–20 minutes each

Pain 3 — Manual Librarian Overhead

Declaring a holiday required a librarian to manually go through the booking register (or spreadsheet) and cancel each booking individually. For a busy day with 50–100 bookings, this could take over an hour. Seat additions or removals required similar manual reconciliation.

- Holiday cancellations took 1–2 hours of manual work
- Seat removals left ghost bookings with no valid seat assignment
- No audit trail for why a booking was cancelled

03 USER PERSONAS

Aryan Mehta, 2nd Year CS — Student

Tech Context: Heavy smartphone user; comfortable with web apps; uses BITS portal daily

Goals	Pain Points
• Reserve a seat and reference book in one booking	• Walks to library to find seats full
• Check real-time availability before going to library	• Can't plan group study — no group booking
• View current and past booking history	• Missed study sessions due to unannounced holidays

Dr. Priya Nair — Librarian

Tech Context: Uses desktop browser; expects fast dashboard; not technical but data-savvy

Goals	Pain Points
• View live seat occupancy by room and hour	• Manual holiday cancellation consumed entire mornings
• Declare holidays with one click	• No single view of occupancy across all library rooms
• See which books are most borrowed	• Couldn't identify underutilised sections for reallocation

Srinivas Rao — System Admin

Tech Context: SQL-comfortable; uses pgAdmin; manages schema changes and procedure deployments

Goals	Pain Points
• Add/remove seats without breaking existing bookings	• Removing a seat caused ghost bookings — no auto-fix
• Register and validate student email domains	• No audit log on trigger executions
• Maintain book inventory with ISBN-level tracking	• Manual email domain validation was error-prone

04 FUNCTIONAL REQUIREMENTS

FR-01 | Available Seat Query

Actor: **Student** Priority: **P0**

The system shall return a list of all seats (seat_id, location, seat_no) that have no booking overlapping with the requested [start_time, end_time] window.

Acceptance Criteria:

- Result must be returned in < 500ms for up to 500 concurrent seat records
- Time window parameters are mandatory; missing params return HTTP 400
- Result is sorted by location then seat_no

FR-02 | Available Book Query

Actor: **Student** Priority: **P0**

The system shall return all books not currently assigned to any booking overlapping the requested time window. Matching is done via JOIN on BookingBook_ID and booking tables.

Acceptance Criteria:

- Returns b_id, b_name, authors, isbn, pub, category
- A book with ISBN X is available if its b_id is not in any overlapping BookingBook_ID row
- Multiple copies of same ISBN shown individually if independently available

FR-03 | Booking Conflict Detection

Actor: **Student** Priority: **P0**

Before confirming any new booking, the system shall call check_booking_conflict() to verify the student has no existing booking that overlaps with the requested slot.

Acceptance Criteria:

- Function returns BOOLEAN — TRUE means safe to proceed
- Overlapping is defined by PostgreSQL OVERLAPS operator on (start_time, end_time)
- Conflict check must be called before INSERT into booking — enforced at API layer

FR-04 | Booking History

Actor: **Student** Priority: **P0**

Authenticated students can retrieve a full, ordered history of their bookings including seat details and any books borrowed during each session.

Acceptance Criteria:

- Results ordered by start_time DESC
- Includes: seat_no, location, start_time, end_time, book_name, isbn, authors, publisher
- Supports optional date-range filter parameters

FR-05 | Current Booking

Actor: **Student** Priority: **P0**

Returns the student's most recent booking that is still active (end_time > NOW()). Handles students with no active booking gracefully.

Acceptance Criteria:

- Returns HTTP 204 if no active booking exists
- Uses subquery to find latest start_time — not all bookings
- Includes LEFT JOINs to handle bookings without a book reservation

FR-06 | Holiday Declaration

Actor: **Librarian** Priority: **P1**

Authenticated librarians can declare the next calendar day as a holiday, triggering an atomic deletion of all bookings with start_time::date = current_date + 1.

Acceptance Criteria:

- All deletions occur in a single transaction — atomic all-or-nothing
- Returns count of cancelled bookings in response body
- Action is logged with timestamp and librarian_id in audit_log table

FR-07 | Student Registration

Actor: **Admin** Priority: **P0**

New students can only be registered via a stored procedure that validates the email domain matches %@hyderabad.bits-pilani.ac.in before INSERT.

Acceptance Criteria:

- Invalid email domain returns specific error: 'Invalid institutional email'
- Duplicate student_id or email raises unique constraint violation
- Password stored as bcrypt hash — never plaintext

05 DATABASE DESIGN

Schema Overview

LibSeat uses a normalised relational schema with five core tables. Foreign key constraints enforce referential integrity; composite keys prevent logical duplicates; and CHECK constraints validate business rules at the database level.

Table	Primary Key	Foreign Keys	Key Constraints
student	student_id (INT)	—	email LIKE '%@hyderabad.bits-pilani.ac.in'
seat	seat_id (INT)	—	seat_no, location UNIQUE together
book	b_id (INT)	—	isbn, b_name, authors, pub, category
booking	(student_id, start_time)	student_id → student seat_id → seat	end_time > start_time
bookingbook_id	(student_id, book_id, start_time)	student_id + start_time → booking book_id → book	end_time > start_time

Key Design Rationale

- **Composite PK on booking: (student_id, start_time):** A student cannot start two bookings at the same time. This is a business rule enforced at the schema level — no application-layer guard needed.
- **Composite PK on bookingbook_id: (student_id, book_id, start_time):** Prevents the same book being allocated to the same student twice in the same slot. ISBN-level uniqueness is handled by trigger logic, not schema.
- **Seat deletion handled by BEFORE DELETE trigger, not ON DELETE CASCADE:** CASCADE would silently destroy booking records. The trigger instead tries to reassign — only cancelling bookings when no alternative seat exists. This preserves student intent.
- **email CHECK constraint in student table:** Enforced at DB level so that even a direct INSERT via psql or pgAdmin cannot bypass institutional email validation — a security guarantee no application layer alone can provide.

06 POSTGRESQL OBJECTS

Triggers

T-01 — before_seat_deletion

Event: **BEFORE DELETE ON seat · FOR EACH ROW**

Fires BEFORE DELETE on each seat row. Attempts to reassign all affected bookings to any free seat. Bookings that cannot be reassigned (no alternative free seat exists) are deleted.

- For each booking using the deleted seat
- Find a seat NOT currently booked during that booking's window
- If found: UPDATE booking SET seat_id = new_seat
- If not found: DELETE booking where seat_id = OLD.seat_id
- RETURN OLD — allows the original DELETE to proceed

T-02 — before_book_deletion

Event: **BEFORE DELETE ON book · FOR EACH ROW**

Fires BEFORE DELETE on each book row. Attempts to reassign BookingBook_ID rows to another physical copy of the same ISBN that is free during the affected booking window.

- For each BookingBook_ID using the deleted book
- Find another book with same ISBN not booked during that window
- If found: UPDATE bookingbook_id SET book_id = new_book
- If not found: DELETE bookingbook_id where book_id = OLD.b_id
- RETURN OLD — allows the original DELETE to proceed

Functions

F-01 — check_booking_conflict

```
check_booking_conflict(sstudent_id INT, sstart_time TIMESTAMP, send_time TIMESTAMP) → BOOLEAN
```

Uses the PostgreSQL OVERLAPS operator to check if the student already has a booking whose [start_time, end_time) overlaps with the requested slot. Returns TRUE if the slot is safe to book.

Returns: TRUE = no conflict, safe to proceed | FALSE = overlap exists, reject booking

F-02 — get_seat_count

```
get_seat_count(location_name VARCHAR) → INT
```

Returns a count of all seats registered under a given location. Used internally by get_next_available_seat() to generate the search range.

Returns: Integer count of seats in the specified location

F-03 — get_next_available_seat

```
get_next_available_seat(location_name VARCHAR) → INT
```

Uses generate_series(1, seat_count+1) to produce a candidate set of seat numbers, then returns the lowest candidate NOT already in the seat table for that location. Supports auto-numbering when a new seat is added.

Returns: Integer: the next available seat_no in the location

Stored Procedure

`declare_holiday_and_delete_bookings()`

A LANGUAGE plpgsql procedure called by the librarian portal to cancel all bookings for the next calendar day in a single atomic transaction. Uses **current_date + 1** to target tomorrow's date without requiring a parameter — simplifying the librarian interface to a single button click. All deletions are rolled back together if any part of the operation fails.

Email Validation Procedure

Student registration is gated by a procedure that checks the supplied email against the pattern **'%@hyderabad.bits-pilani.ac.in'** using a LIKE clause before executing the INSERT. This enforces institutional access control at the database level, independent of the application layer. An attempt to register a non-BITS email raises a custom exception with a descriptive message.

07 API SPECIFICATION

All endpoints are implemented as Flask route handlers. Authentication is session-based with role differentiation between student and librarian tokens. All timestamps follow ISO 8601 format.

Method	Endpoint	Role	Parameters	Response
GET	/available-seats	Student	start_time, end_time (query)	200: [{seat_id, location, seat_no}]
GET	/available-books	Student	start_time, end_time (query)	200: [{b_id, b_name, authors, isbn, pub, category}]
GET	/check-booking-conflict	Student	student_id, start_time, end_time	200: {conflict: bool}
GET	/booking-history	Student	student_id (session)	200: [{seat_no, location, start_time, end_time, book_name, isbn, authors, pub}]
GET	/current-booking	Student	student_id (session)	200: {seat_no, location, start_time, end_time, book details} | 204: no booking
POST	/declare-holiday	Librarian	(none — uses current_date+1)	200: {cancelled_count: N} | 403: not librarian
POST	/register-student	Admin	name, email, student_id, password	201: created | 400: invalid email domain

08 LIBRARIAN ANALYTICS

The librarian dashboard exposes two analytical query patterns identified during requirements. Both are implemented as direct SQL queries in Flask routes, with results rendered as tabular and chart-ready JSON for the dashboard frontend.

Query 1 — Hourly Seat Occupancy

For each 1-hour interval across the day, returns the total number of booking records whose [start_time, end_time) window includes that hour. Enables librarians to identify peak hours and staff accordingly. Results are displayed as a bar chart on the dashboard.

SQL — Hourly Occupancy

```
SELECT
  date_trunc('hour', generate_series) AS hour_slot,
  COUNT(b.student_id) AS seats_occupied
FROM generate_series(
  date_trunc('day', NOW()),
  date_trunc('day', NOW()) + INTERVAL '23 hours',
  INTERVAL '1 hour'
) AS generate_series
LEFT JOIN booking b
  ON generate_series >= b.start_time
  AND generate_series <  b.end_time
GROUP BY hour_slot ORDER BY hour_slot;
```

Query 2 — Room Occupancy Percentage

For each hour and each room (location), calculates the percentage of seats that are currently occupied. Enables room-level capacity planning and identification of underutilised spaces. The denominator is the total seat count per location from the seat table.

SQL — Room Occupancy %

```
SELECT
  s.location,
  date_trunc('hour', generate_series) AS hour_slot,
  ROUND(
    COUNT(b.student_id) * 100.0 / COUNT(s.seat_id), 1
  ) AS occupancy_pct
FROM generate_series(...) AS generate_series
CROSS JOIN seat s
LEFT JOIN booking b
  ON b.seat_id = s.seat_id
  AND generate_series >= b.start_time
  AND generate_series <  b.end_time
GROUP BY s.location, hour_slot
ORDER BY hour_slot, s.location;
```

Last-Seat Alert

A trigger-adjacent feature: when the count of available seats in a given location drops to exactly 1 during a booking insertion, the system emits an alert flag in the API response. The librarian dashboard polls this endpoint every 60 seconds and displays a prominent banner for the affected location. This is implemented as a database NOTIFY / application LISTEN pattern — not polling the full availability query.

09 SUCCESS METRICS

Student Experience

Metric	Definition	Target	Priority
Booking Completion Rate	% of initiated bookings that complete successfully	**> 90%**	P
Double-Booking Incidents	Confirmed conflict bookings in database	**0 per month**	P
Avg. Time to Find Available Seat	Session start to seat result displayed	**< 30 seconds**	P
Booking History Load Time	History page load for 100+ records	**< 1 second**	S

Librarian Operations

Metric	Definition	Target	Priority
Holiday Cancellation Time	Time from button click to all bookings deleted	**< 5 seconds**	P
Manual Intervention Rate	% of booking issues requiring librarian action	**< 5%**	P
Dashboard Refresh Time	Occupancy stats query execution time	**< 2 seconds**	P
Last-Seat Alert Latency	Time from 2nd-to-last seat booked to alert shown	**< 60 seconds**	S

System Health

Metric	Definition	Target	Priority
Trigger Success Rate	% of seat/book deletions handled without error	**> 99.9%**	P
Auto-Reassignment Rate	% of affected bookings auto-reassigned on deletion	**> 80%**	P
Query Response Time (P95)	95th percentile DB query time	**< 500ms**	P
Email Validation Rejection Rate	% of invalid-domain registration attempts caught	**100%**	P

Anti-Metrics

- Total bookings per day — we want accurate bookings, not high volume
- Database size — growth is expected; unconstrained growth is the risk to watch
- Number of triggers fired — high is good; it means the protection is working

10 ROADMAP & APPENDIX

Delivery Roadmap

Phase 1 — Core Booking System

Timeline: Week 1–3 | **Goal:** Validate FR-01 through FR-05 end-to-end

Feature / Deliverable	Priority
Student registration with email validation procedure	P0
Seat availability query (/available-seats)	P0
Book availability query (/available-books)	P0
Booking conflict detection function	P0
Booking creation + history endpoints	P0

Phase 2 — Automation Layer

Timeline: Week 4–6 | **Goal:** Zero-touch operations on resource removal

Feature / Deliverable	Priority
Seat deletion trigger with auto-reassign	P0
Book deletion trigger with ISBN-match reassign	P0
Holiday declaration stored procedure	P0
Current booking endpoint	P0
Last-seat alert mechanism (NOTIFY/LISTEN)	P1

Phase 3 — Analytics & Dashboard

Timeline: Week 7–8 | **Goal:** Librarian intelligence and visibility

Feature / Deliverable	Priority
Hourly occupancy stats query + API endpoint	P0
Room occupancy % query + API endpoint	P0
Librarian web dashboard (React frontend)	P1
Audit log table for trigger-driven actions	P1
CSV export of booking and occupancy reports	P2

Open Questions

#	Question	Trade-off

OQ-01	**Notification on Cancellation**	Should students receive an email/SMS when their booking is auto-cancelled by a trigger or holiday procedure? Requires external SMTP integration.
OQ-02	**Group Booking**	The current schema books exactly one seat per student. Should a future version support group seat reservations under a single student_id (group leader)?
OQ-03	**Booking Duration Limits**	Should there be a maximum booking duration enforced by a CHECK constraint (e.g., no single booking > 6 hours)? Currently unlimited.
OQ-04	**Waitlist Queue**	When a seat becomes available (booking cancelled), should a waitlist mechanism automatically notify the next student in queue?

Key Assumptions

- All students have a valid @hyderabad.bits-pilani.ac.in email address at time of registration
- A booking always involves exactly one seat; book reservation is optional within the booking
- The PostgreSQL server has sufficient connection pooling for peak concurrent usage (estimated 200 students)
- The librarian portal runs on a trusted internal network — no public internet exposure in V1
- Physical book copies are individually catalogued with unique b_id even when sharing an ISBN